

AI-SERVICE · DOCUMENTACIÓN TÉCNICA

FinSightAI AI-Service

FastAPI · Python · Machine Learning · Finsi

Microservicio de inteligencia, análisis financiero
y asistencia conversacional



HACKATHON ONE — PROYECTOS G9

G9 Team 29 · TwentyNine Devs

Alura · Oracle · No Country | Versión 1.0.0 | Agosto 2026

1. Resumen ejecutivo

El **AI-Service** es el microservicio inteligente de FinSightAI. Centraliza la clasificación automática de transacciones, el análisis del perfil financiero, el procesamiento de archivos CSV, la generación de recomendaciones estructuradas, la consulta conversacional mediante **Finsi** y la gestión de metas financieras expuesta al resto de la plataforma.

La API está implementada con **FastAPI** y **Pydantic**. Los modelos de Machine Learning se serializan con Joblib y fueron entrenados con scikit-learn. El asistente integra proveedores LLM y construye contexto financiero específico según la intención de la consulta.

Objetivos principales

- Transformar datos financieros y transacciones en información estructurada y accionable.
- Clasificar gastos e ingresos y asignar categoría y subcategoría.
- Estimar perfil financiero, riesgo, Financial Score y métricas derivadas.
- Generar recomendaciones explicables con diagnóstico, acción, objetivo y evidencia.
- Permitir consultas financieras conversacionales sin delegar al LLM los cálculos determinísticos.
- Procesar movimientos importados por CSV y normalizarlos para la plataforma.

Alcance

Este documento cubre exclusivamente el AI-Service y sus interfaces con los consumidores externos. El Frontend y el Backend Spring Boot se consideran sistemas externos y cuentan con documentación independiente.

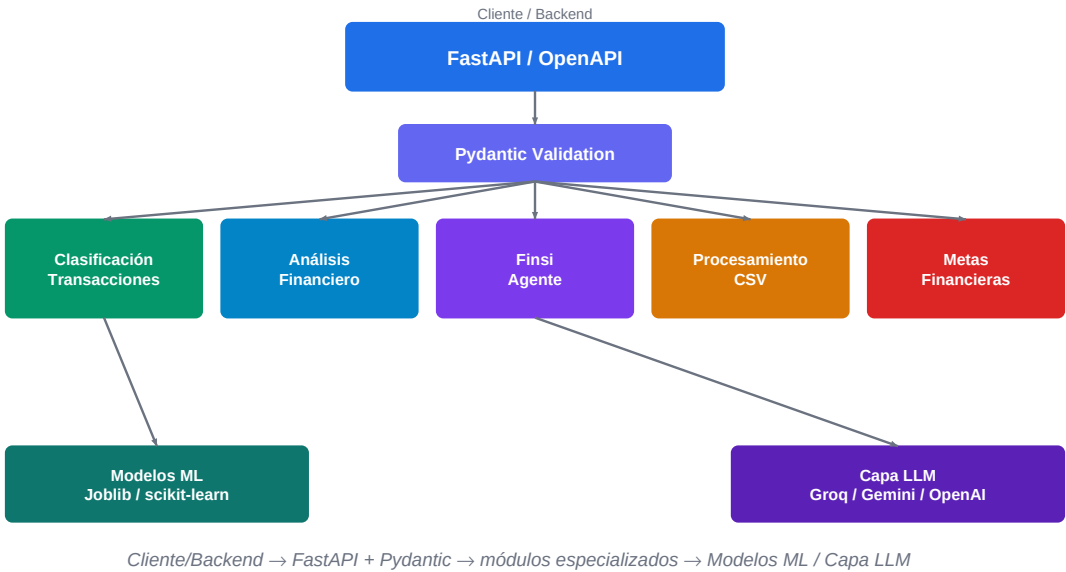
Principios de diseño

Principio	Aplicación en AI-Service
Separación de responsabilidades	Routers, schemas, servicios, modelos ML y agente se mantienen en módulos diferenciados.
Validación temprana	Pydantic y validadores de dominio rechazan entradas incoherentes antes del análisis.
Resultados explicables	Score, riesgo, métricas, fortalezas, oportunidades y evidencia acompañan las recomendaciones.
Contexto mínimo	Finsi selecciona los datos financieros necesarios según la intención detectada.
Contrato API	FastAPI expone OpenAPI y Swagger UI automáticamente.

2. Arquitectura del AI-Service

El AI-Service recibe solicitudes HTTP desde el Backend o consumidores autorizados. Los routers FastAPI delegan la lógica a módulos especializados: predicción, análisis de perfil, procesamiento CSV, agente conversacional y metas.

Diagrama de componentes



Estructura interna relevante

Módulo	Responsabilidad
app/api	Endpoints: health, category, csv_import, analysis, agent y goals.
app/schemas	Contratos Pydantic de request/response.
app/prediction.py	Carga y ejecución del clasificador de transacciones.

Módulo	Responsabilidad
app/profile.py	Análisis financiero, score, perfil y recomendaciones.
app/services/csv_processor.py	Validación y transformación de CSV.
app/services/agent	Orquestación de Finsi, intención, contexto, reglas y consultas.
app/services/llm	Integraciones con proveedores LLM (Groq, Gemini, OpenAI).
app/services/goals	Lógica de metas, reservas, liberaciones y resumen.
models / artefactos	Modelos serializados, metadata y métricas.
tests	Pruebas automatizadas de rutas, agente, contexto, CSV y recomendaciones.

3. Stack tecnológico y ejecución

Capa	Tecnología / versión
API	FastAPI 0.141.1 · Uvicorn 0.52.1
Validación	Pydantic 2.13.4 · pydantic-settings 2.14.2
Datos	Pandas 2.2.3 · NumPy 2.5.1
Machine Learning	scikit-learn 1.6.1 · Joblib 1.4.2
LLM	Groq 1.6.0 · Google GenAI 2.16.0 · cliente OpenAI
RAG / vectores	FAISS CPU 1.15.0
Testing	Pytest 9.1.1 · HTTPX 0.28.1 · pytest-asyncio 1.4.0
Containerización	Docker · python:3.12-slim

Ejecución local

python -m uvicorn app.main:app --reload
Swagger UI: http://127.0.0.1:8000/docs · OpenAPI: http://127.0.0.1:8000/openapi.json

Containerización y seguridad

El Dockerfile instala dependencias, copia app, models y data, expone el puerto 8000 y ejecuta Uvicorn. Credenciales por variables de entorno. CORS desde settings.

Estado del servicio

GET /health → 200 OK

```
{
  "status": "ok",
  "service": "FinSightAI AI Service",
  "version": "1.0.0",
  "environment": "development",
  "components": {
    "machine_learning": { "status": "configured", "configured": true },
    "groq": { "status": "configured", "configured": true },
    "gemini": { "status": "configured", "configured": true }
  }
}
```

4. Machine Learning

Dos modelos principales serializados con Joblib (random_state = 42). Entrenados con datos sintéticos para el MVP; en producción se recomienda reentrenar con datos reales.

4.1 Clasificador de transacciones

Feature: descripcion_limpia. 16 clases: Alimentación, Compras, Deudas, Educación, Entretenimiento, Impuestos, Otros, Otros ingresos, Reintegro, Salario, Salud, Servicios, Transferencia recibida, Transporte, Venta y Vivienda.

Evaluación	Accuracy	Precision macro	Recall macro	F1 macro
Hold-Out	1.0000	1.0000	1.0000	1.0000
CV agrupada	0.9967	0.9858	0.9771	0.9767

4.2 Clasificador de perfil financiero

Perfiles: **Saludable**, **En observación** o **En riesgo**. Features: endeudamiento, % gastos/ingresos, cantidad de transacciones, recurrencias, transacciones grandes, diversidad de categorías y ratios de deuda, gasto y ahorro.

Evaluación	Accuracy	Precision macro	Recall macro	F1 macro
Hold-Out	0.8760	0.8771	0.8885	0.8817
Validación cruzada	0.8960	0.8992	0.9053	0.9013

Métricas adecuadas para el MVP. El clasificador de transacciones alcanza rendimiento óptimo en hold-out por el diseño del dataset sintético; el de perfil muestra métricas sólidas.

5. Contrato de API

Contrato OpenAPI con Swagger UI. Rutas del servicio:

Método	Endpoint	Descripción
GET	/	Estado básico y acceso a documentación.
GET	/health	Salud del servicio y componentes configurados.
POST	/predict/category	Clasifica una transacción.
POST	/csv/procesar	Valida y transforma un CSV de movimientos.
POST	/analysis	Ejecuta un análisis financiero con datos provistos.
GET	/analysis/users/{usuario_id}	Obtiene o genera análisis de un usuario existente.
POST	/agent/chat	Consulta conversacional a Finsi.
POST	/agent/chat/stream	Consulta Finsi mediante Server-Sent Events.
GET	/goals/users/{usuario_id}	Lista metas; admite filtro por estado.
GET	/goals/{goal_id}	Consulta una meta individual.
POST	/goals	Crea una meta financiera.
PATCH	/goals/{goal_id}	Actualiza una meta.
POST	/goals/{goal_id}/reserve	Reserva dinero para una meta.
POST	/goals/{goal_id}/release	Libera dinero reservado.
DELETE	/goals/{goal_id}	Cancela una meta financiera.
GET	/goals/users/{usuario_id}/summary	Obtiene resumen agregado de metas.

Códigos de respuesta

Éxito: **200**. Reglas de negocio: **400**. No encontrado: **404**. Validación schema/CSV: **422**.

6. Ejemplos de Request / Response

6.1 Clasificación automática

POST /predict/category

```
{
  "descripcion": "Pago de alquiler",
  "monto": 850,
  "fecha": "2026-07-21",
  "medio_pago": "Transferencia bancaria",
  "recurrente": "Sí"
}
```

200 OK

```
{
  "tipo_transaccion": "GASTO",
  "categoria_predicha": "Vivienda",
  "subcategoria_predicha": "Alquiler",
  "confianza": 0.99,
  "metodo_clasificacion": "regla_exacta",
  "advertencias": [],
  "modelo_version": "8.1.0"
}
```

6.2 Análisis financiero — escenario saludable

POST /analysis

```
{
  "usuario_id": "USR1013",
  "financial_score": 90,
  "score_status": "Excelente",
  "nivel_riesgo": "Bajo",
  "perfil_financiero": "Saludable",
  "confianza_perfil": 0.9982,
  "modelo_version": "7.1.0"
}
```

Gastos 53.6 % de ingresos, deuda 7.8 %, capacidad de ahorro 38.6 %. Genera fortalezas, oportunidades y recomendaciones estructuradas.

6.3 Análisis por usuario — escenario crítico

GET /analysis/users/USR1000

```
{
  "financial_score": 20,
  "score_status": "Crítico",
  "nivel_riesgo": "Crítico",
  "perfil_financiero": "En riesgo",
  "confianza_perfil": 0.9977
}
```

Gastos 85.6 %, deuda 33.3 %, ahorro -18.9 %. Recomendaciones de prioridad alta y media.

7. CSV y normalización de movimientos

POST /csv/procesar recibe usuario_id y un archivo multipart. Respuesta en tres bloques: **usuario** (métricas), **transacciones** (movimientos normalizados) y **resumen** (agregados). Ejemplo de validación: 33 transacciones en 3 meses, USD 10 300 ingresos / USD 6 328.29 gastos.

8. Finsi — agente financiero conversacional

Capa conversacional sobre datos estructurados. Detección de intención, consultas determinísticas, reglas, contexto mínimo y proveedores LLM. El FinancialContextBuilder selecciona solo los campos necesarios según la intención; los cálculos críticos permanecen fuera del LLM.

Chat tradicional

POST /agent/chat

```
{
  "usuario_id": "USR1000",
  "question": "Cómo están mis finanzas"
}
```

200 OK

```
{
  "answer": "***Resumen**\nTu situación financiera actual es crítica...",
  "provider": "groq"
}
```

Streaming SSE

POST /agent/chat/stream · Content-Type: text/event-stream

```
data: { "status": "step", "message": "Finsi está revisando tus datos financieros..." }
data: { "status": "done", "answer": "La situación financiera muestra...", "provider": "groq" }
```

9. Recomendaciones financieras

Objetos estructurados para que el Frontend represente prioridad, diagnóstico y próximos pasos.

Campo	Función
id	Identificador estable de la recomendación.
tipo	Familia de recomendación financiera.
perfil	Perfil financiero asociado.
prioridad	alta, media o sugerencia.
titulo	Nombre legible para UI.
diagnostico	Explicación basada en los datos.

Campo	Función
accion	Acción sugerida.
objetivo	Meta cuantificable cuando corresponde.
evidencia	Valores numéricos o categóricos que justifican la recomendación.
pregunta_finsi	Prompt contextual para continuar el análisis con Finsi.
advertencia	Límite de alcance y recomendación de asesoramiento profesional.

10. Metas financieras

CRUD de metas, reservas y liberaciones. Expone monto objetivo, reservado, restante, progreso, fecha objetivo, reserva mensual sugerida, estado y timestamps.

GET /goals/users/USR0001

```
{
  "nombre": "Saldar Deuda de Credito",
  "categoria": "DEUDA",
  "monto_objetivo": 40000,
  "monto_reservado": 10000,
  "monto_restante": 30000,
  "progreso": 25,
  "reserva_mensual_sugerida": 6000,
  "estado": "ACTIVA"
}
```

DELETE /goals/{goal_id} cancela. /reserve y /release modifican el reservado.

11. Casos de uso

ID	Caso de uso	Actor	Interfaz
UC-01	Verificar salud del servicio	Backend / ops	GET /health
UC-02	Clasificar una transacción	Backend	POST /predict/category
UC-03	Analizar información financiera	Backend	POST /analysis
UC-04	Analizar un usuario por ID	Backend	GET /analysis/users/{id}
UC-05	Procesar movimientos CSV	Backend / usuario	POST /csv/procesar
UC-06	Consultar a Finsi	Backend / usuario	POST /agent/chat
UC-07	Consultar a Finsi en streaming	Frontend / Backend	POST /agent/chat/stream
UC-08	Gestionar metas financieras	Backend	/goals...

UC-02: normaliza, aplica reglas/modelo → tipo, categoría, confianza, versión. **UC-03:** valida métricas, estima perfil, genera score y recomendaciones. **UC-05:** valida CSV, clasifica y agrega. **UC-06/07:** intención + contexto mínimo + respuesta (SSE). **UC-08:** CRUD, reserve/release, resumen.

12. Validación, errores y calidad

Coherencia matemática en análisis: $\text{nivel_endeudamiento} \approx \text{deuda}/\text{ingreso}$, $\text{porcentaje_gastos_ingreso} \approx \text{gasto}/\text{ingreso}$, $\text{ahorro_mensual_estimado} \approx \text{ingreso} - \text{gasto} - \text{deuda}$. «Ahorro» no se admite como categoría de gasto.

Código	Uso
200	Operación exitosa.
400	Regla de negocio, incoherencia o solicitud inválida.
404	Usuario o meta no encontrada.
422	Error de validación de schema o CSV.

Suite Pytest: agente, intents, CSV, contexto, recomendaciones, seguridad y timezone. Artefactos Joblib requieren la versión de scikit-learn de requirements.txt.

13. Limitaciones y consideraciones

- Modelos con datos sintéticos; producción requiere datos reales y monitoreo.
- Recomendaciones educativas; no sustituyen asesoramiento profesional.
- Respuestas generativas dependen del LLM; cálculos críticos son determinísticos.
- Unificar versionado de modelos de categoría y perfil en futuras iteraciones.

14. Trazabilidad con los requisitos del Hackathon

Requisito	Cobertura en AI-Service
Clasificación automática de gastos	/predict/category + clasificador de transacciones.
Análisis del comportamiento financiero	/analysis y /analysis/users/{usuario_id}.
Perfil financiero	Saludable / En observación / En riesgo.
Indicadores financieros	Financial Score, riesgo, ratios, categorías principales.
Recomendaciones personalizadas	Objetos estructurados con diagnóstico, acción, objetivo y evidencia.
API REST	FastAPI + OpenAPI + Swagger.
Procesamiento CSV	/csv/procesar.
Asistente inteligente	Finsi: chat y SSE.
Metas financieras	CRUD, reservas, liberaciones y resumen.
Pruebas automatizadas	Suite Pytest incluida en el repositorio.
Containerización	Dockerfile incluido.

15. Conclusión

El AI-Service es la capa de inteligencia de FinSightAI: combina Machine Learning, reglas financieras, validación, procesamiento de datos, recomendaciones explicables e interfaz conversacional contextual. La separación modular permite que el Backend consuma capacidades sin acoplarse a modelos o proveedores LLM.

Prioridades post-MVP: validar con datos reales, consolidar versionado, reforzar observabilidad y autenticación entre servicios, y mantener pruebas reproducibles.

FinSightAI · TwentyNine Devs · G9 Team 29
Hackathon ONE — Proyectos G9 · Alura + Oracle + No Country